

```
import os
import re
import logging
import tempfile
import asyncio
from pathlib import Path
from typing import Optional

import yt_dlp
from dotenv import load_dotenv
from telegram import Update
from telegram.constants import ChatAction
from telegram.ext import (
    Application,
    CommandHandler,
    MessageHandler,
    ContextTypes,
    filters,
)

load_dotenv()

BOT_TOKEN = os.getenv("BOT_TOKEN")
MAX_FILE_SIZE_MB = 50 # лимит Telegram Bot
API на прямую загрузку файла

logging.basicConfig(
    format="%(asctime)s - %(name)s - %(
levelname)s - %(message)s",
    level=logging.INFO,
```

```

)
logger = logging.getLogger("SaveNovaBot")

# Поддерживаемые площадки: TikTok,
Instagram (Reels), YouTube Shorts
URL_PATTERN = re.compile(
    r"(https?:/(?:www\.|vm\.|vt\.|m\.))?"
    r"(?:tiktok\.com|instagram\.com|youtube\.com|
youtu\.be)"
    r"/[^\s]+)",
    re.IGNORECASE,
)

WELCOME_TEXT = (
    "👋 Привет! Я *SaveNovaBot*.\n\n"
    "Пришли мне ссылку на видео из:\n"
    "🎵 TikTok\n"
    "📷 Instagram Reels\n"
    "📺 YouTube Shorts\n\n"
    "И я скачаю его без водяного знака и
подписей 🚀"
)

HELP_TEXT = (
    "Просто пришли ссылку на видео из
TikTok, Instagram Reels или "
    "YouTube Shorts – я отправлю тебе файл
без водяного знака.\n\n"
    "Команды:\n"
    "/start – приветствие\n"
    "/help – эта справка"
)

```

```
async def start(update: Update, context:
ContextTypes.DEFAULT_TYPE) -> None:
    await
    update.message.reply_text(WELCOME_TEXT,
    parse_mode="Markdown")
```

```
async def help_command(update: Update,
context: ContextTypes.DEFAULT_TYPE) ->
None:
    await
    update.message.reply_text(HELP_TEXT)
```

```
def _extract_url(text: str) ->
Optional[str]:
    match = URL_PATTERN.search(text)
    return match.group(1) if match else
None
```

```
def _download_video(url: str, out_dir: str)
-> Path:
    """Блокирующая загрузка через yt-dlp.
    Вызывается в отдельном потоке."""
    outtmpl = os.path.join(out_dir, "%
(id)s.%(ext)s")

    ydl_opts = {
        "outtmpl": outtmpl,
        "format": "mp4/best",
        "merge_output_format": "mp4",
        "quiet": True,
```

```

        "no_warnings": True,
        "noplaylist": True,
        "max_filesize": MAX_FILE_SIZE_MB *
1024 * 1024,
    }

    with yt_dlp.YoutubeDL(ydl_opts) as ydl:
        info = ydl.extract_info(url,
download=True)
        filename =
ydl.prepare_filename(info)
        return Path(filename)

async def handle_message(update: Update,
context: ContextTypes.DEFAULT_TYPE) ->
None:
    text = update.message.text or ""
    url = _extract_url(text)

    if not url:
        await update.message.reply_text(
            "Не вижу ссылку на TikTok,
Instagram или YouTube Shorts 🙄"
        )
        return

    status_msg = await
update.message.reply_text("⌚ Скачиваю
видео...")
    await context.bot.send_chat_action(
        chat_id=update.effective_chat.id,
        action=ChatAction.UPLOAD_VIDEO
    )

```

```

        with tempfile.TemporaryDirectory() as
tmp_dir:
            try:
                file_path = await
asyncio.to_thread(_download_video, url,
tmp_dir)
            except yt_dlp.utils.DownloadError
as e:
                logger.warning("Download failed
for %s: %s", url, e)
                await status_msg.edit_text(
                    "❌ Не получилось скачать
это видео. Возможно, оно приватное, "
                    "удалено или ссылка
неверна."
                )
                return
            except Exception:
                logger.exception("Unexpected
error downloading %s", url)
                await status_msg.edit_text("❌
Произошла ошибка при скачивании.")
                return

            if not file_path.exists():
                await status_msg.edit_text("❌
Файл не найден после скачивания.")
                return

            size_mb = file_path.stat().st_size
/ (1024 * 1024)
            if size_mb > MAX_FILE_SIZE_MB:
                await status_msg.edit_text(

```

```

        f"❌ Видео весит
        {size_mb:.1f} МБ — это больше лимита в "
        f"{MAX_FILE_SIZE_MB} МБ для
        прямой отправки ботом."
    )
    return

    try:
        with open(file_path, "rb") as
        video_file:
            await
            update.message.reply_video(
                video=video_file,
                caption="✅ Готово! Без
                водяного знака — @SaveNovaBot",

            supports_streaming=True,
            )
            await status_msg.delete()
        except Exception:
            logger.exception("Failed to
            send video for %s", url)
            await status_msg.edit_text("❌
            Не получилось отправить видео.")

def main() -> None:
    if not BOT_TOKEN:
        raise RuntimeError(
            "Не задан BOT_TOKEN. Создай
            файл .env (по примеру .env.example) "
            "и укажи в нём
            BOT_TOKEN=токен_от_BotFather"
        )

```

```
    app =
Application.builder().token(BOT_TOKEN).build()

    app.add_handler(CommandHandler("start",
start))
    app.add_handler(CommandHandler("help",
help_command))

app.add_handler(MessageHandler(filters.TEXT
& ~filters.COMMAND, handle_message))

    logger.info("SaveNovaBot запущен")

app.run_polling(allowed_updates=Update.ALL_
TYPES)

if __name__ == "__main__":
    main()
```